3주차 / BTNode.java

알고리즘 · 코드 모음

```java
package week_03;

public class BTNode {
    String data;
    BTNode Lchild;
    BTNode Rchild;

    public BTNode() {
    }

    public BTNode(String dt) {
        data = dt;
        Lchild = null;
        Rchild = null;
    }

    public BTNode(BTNode lc, String dt, BTNode rc) {
        data = dt;
        Lchild = lc;
        Rchild = rc;

    }
}
```

3주차 / BinaryTree.java

알고리즘 · 코드 모음

```java
package week_03;

public class BinaryTree {
    BTNode root;

    public BinaryTree() {
        // 코드 작성
        root = null;

    }

    public BinaryTree(BinaryTree ltree, String data, BinaryTree rtree) {
        // 코드 작성
        root = new BTNode(ltree.root,data,rtree.root);
    }

    public boolean isEmpty() {
        // 코드 작성
        return root == null;
    }

    public BinaryTree leftSubTree() {
        // 코드 작성
        BinaryTree tmp = new BinaryTree();
        if(root != null) {
            tmp.root = root.Lchild;
        }
        return tmp;
    }

    public BinaryTree rightSubTree() {
        // 코드 작성
        BinaryTree tmp = new BinaryTree();
        if(root != null) {
            tmp.root = root.Rchild;
        }
        return tmp;
    }

    public String rootData() {
        // 코드 작성
        if(root == null) {
            System.out.println("빈루트");
            return null;
        }
        return root.data;
    }
```

```
48
49      public int calculate() {
50          // 코드 작성
51          return theCalculate(root);
52      }
53
54      private int theCalculate(BTNode t) {
55          // 코드 작성
56          if (t.Lchild == null && t.Rchild == null) {
57              return Integer.parseInt(t.data);
58          }
59          int left = theCalculate(t.Lchild);
60          int right = theCalculate(t.Rchild);
61
62          switch (t.data) {
63              case "+":
64                  return left + right;
65
66              case "-":
67                  return left - right;
68
69              case "*":
70                  return left * right;
71
72              case "/":
73                  return left / right;
74
75              default:
76                  return 0;
77          }
78      }
79
80      public int height() {
81          // 코드 작성
82          return theHeight(root);
83      }
84
85      private int theHeight(BTNode t) {
86          // 코드 작성
87          if(t == null) {
88              return -1;
89          }
90          int leftHeight = theHeight(t.Lchild);
91          int rightHeight = theHeight(t.Rchild);
92
93          return 1 + Math.max(leftHeight, rightHeight);
94      }
```

```
95
96      public void print0() {
97          // 코드 작성
98          print0(root);
99          System.out.println();
100     }
101
102     private void print0(BTNode t) {
103         // 코드 작성
104         if(t==null)return;
105
106         if (t.Lchild == null && t.Rchild == null) {
107             System.out.print(t.data);
108             return;
109         }
110
111         System.out.print("(");
112
113         print0(t.Lchild);
114
115         System.out.print(t.data);
116
117         print0(t.Rchild);
118
119         System.out.print(")");
120     }
121 }
```

3주차 / BinaryTreeTest.java

알고리즘 · 코드 모음

```java
package week_03;

public class BinaryTreeTest {
    public static void main(String args[]) {
        BinaryTree btree;
        BinaryTree ltree;
        BinaryTree rtree;
        BinaryTree current;
        rtree = new BinaryTree(new BinaryTree(), "1", new BinaryTree());
        ltree = new BinaryTree(new BinaryTree(), "2", new BinaryTree());
        btree = new BinaryTree(ltree, "+", rtree);
        ltree = btree;
        rtree = new BinaryTree(new BinaryTree(), "3", new BinaryTree());
        btree = new BinaryTree(ltree, "*", rtree);
        System.out.println("계산 결과 : " + btree.calculate());
        System.out.print("수식 출력 : ");
        btree.print0();
        System.out.println("트리의 높이 : " + btree.height());
    }
}
```

5주차 / BTNode.java

알고리즘 · 코드 모음

```java
package week_05;

public class BTNode {
    String data;
    BTNode Lchild;
    BTNode Rchild;

    public BTNode() {
    }

    public BTNode(String dt) {
        data = dt;
        Lchild = null;
        Rchild = null;
    }

    public BTNode(BTNode lc, String dt, BTNode rc) {
        data = dt;
        Lchild = lc;
        Rchild = rc;
    }
}
```

5주차 / BinaryTree.java

알고리즘 · 코드 모음

```java
package week_05;

public class BinaryTree {
    BTNode root;

    public BinaryTree() {
        root = null;
    }

    public BinaryTree(BinaryTree ltree, String data, BinaryTree rtree) {
        root = new BTNode(ltree.root, data, rtree.root);
    }

    public boolean isEmpty() {
        return root == null;
    }

    public BinaryTree leftSubTree() {
        BinaryTree ltree = new BinaryTree();
        ltree.root = root.Lchild;
        return ltree;
    }

    public BinaryTree rightSubTree() {
        BinaryTree rtree = new BinaryTree();
        rtree.root = root.Rchild;
        return rtree;
    }

    public String rootData() {
        return root.data;
    }

    public void inorder() {
        System.out.println();
        System.out.println("Inorder");
        theInorder(root);
        System.out.println();
        System.out.println("InorderIter");
        inorderIter1();
    }

    public void preorder() {
        System.out.println("preorder");
        System.out.println();
        thePreorder(root);
        System.out.println();
```

```
48          System.out.println("preorderIter");
49          preorderIter();
50      }
51
52      public void postorder() {
53          thePostorder(root);
54      }
55
56      private void theInorder(BTNode t) {
57          // 코드 작성
58           if (t != null) {
59                  theInorder(t.Lchild);
60
61                  System.out.print(t.data);
62
63                  theInorder(t.Rchild);
64              }
65      }
66
67      private void thePreorder(BTNode t) {
68          // 코드 작성
69           if (t != null) {
70                  System.out.print(t.data);
71
72                  thePreorder(t.Lchild);
73
74                  thePreorder(t.Rchild);
75              }
76      }
77
78      private void thePostorder(BTNode t) {
79          // 코드 작성
80          if (t != null) {
81              thePostorder(t.Lchild);
82
83              thePostorder(t.Rchild);
84
85              System.out.print(t.data);
86          }
87      }
88
89      private void inorderIter1() {
90          // 코드 작성
91          Stack stack = new Stack();
92
93          BTNode current = root;
94
```

```
95          while (current != null || !stack.empty()) {
96
97              // 가장 왼쪽 노드까지 이동
98              while (current != null) {
99                  stack.push(current);
100                 current = current.Lchild;
101             }
102
103             // Stack에서 하나 꺼냄
104             current = (BTNode) stack.pop();
105
106             System.out.print(current.data);
107
108             // 오른쪽 서브트리 이동
109             current = current.Rchild;
110         }
111     }
112
113     private void preorderIter() {
114         // 코드 작성
115         if (root == null)
116             return;
117
118         Stack stack = new Stack();
119
120         stack.push(root);
121
122         while (!stack.empty()) {
123
124             BTNode current = (BTNode) stack.pop();
125
126             System.out.print(current.data);
127
128             // Stack은 LIFO이므로
129             // 오른쪽 먼저 왼쪽 나중
130             if (current.Rchild != null)
131                 stack.push(current.Rchild);
132
133             if (current.Lchild != null)
134                 stack.push(current.Lchild);
135         }
136     }
137
138     public void levelorder() {
139         // 코드 작성
140         if (root == null)
141             return;
```

```
142
143         Queue queue = new Queue();
144
145         queue.enqueue(root);
146
147         while (!queue.isEmpty()) {
148
149             BTNode current = (BTNode) queue.dequeue();
150
151             System.out.print(current.data);
152
153             if (current.Lchild != null)
154                 queue.enqueue(current.Lchild);
155
156             if (current.Rchild != null)
157                 queue.enqueue(current.Rchild);
158         }
159     }
160
161     public BinaryTree copy() {
162         // 코드 작성
163         BinaryTree newTree = new BinaryTree();
164
165         newTree.root = theCopy(root);
166
167         return newTree;
168     }
169
170     private BTNode theCopy(BTNode t) {
171         // 코드 작성
172         if (t == null)
173             return null;
174
175         BTNode newNode =
176                 new BTNode(
177                         theCopy(t.Lchild),
178                         t.data,
179                         theCopy(t.Rchild)
180                 );
181
182         return newNode;
183     }
184
185     public boolean equals(BinaryTree tr) {
186         return theEqual(this.root, tr.root);
187     }
188
```

```
189     private boolean theEqual(BTNode s, BTNode t) {
190         // 둘다 널
191         if (s == null && t == null)
192             return true;
193 
194         // 둘 중 하나만 널
195         if (s == null || t == null)
196             return false;
197 
198         // 데이터가 다르면 다른 트리
199         if (!s.data.equals(t.data))
200             return false;
201 
202         // 왼쪽 오른쪽 모두 비교
203         return theEqual(s.Lchild, t.Lchild)
204                 && theEqual(s.Rchild, t.Rchild);
205     }
206 }
```

5주차 / BinaryTreeTest.java

알고리즘 · 코드 모음

```java
package week_05;

public class BinaryTreeTest {
    public static void main(String args[]) {
        BinaryTree btree;
        BinaryTree ltree;
        BinaryTree rtree;
        BinaryTree current;
        rtree = new BinaryTree(new BinaryTree(), "H", new BinaryTree());
        btree = new BinaryTree(new BinaryTree(), "G", rtree);
        ltree = btree;
        btree = new BinaryTree(ltree, "E", new BinaryTree());
        ltree = new BinaryTree(new BinaryTree(), "D", new BinaryTree());
        rtree = btree;
        btree = new BinaryTree(ltree, "B", rtree);
        ltree = new BinaryTree(new BinaryTree(), "F", new BinaryTree());
        current = new BinaryTree(ltree, "C", new BinaryTree());
        ltree = btree;
        rtree = current;
        btree = new BinaryTree(ltree, "A", rtree);
        System.out.println(btree.rootData());
        System.out.println(btree.leftSubTree().rootData());
        System.out.println(btree.rightSubTree().rootData());
        System.out.println(btree.leftSubTree().leftSubTree().rootData());
        System.out.println(btree.rightSubTree().leftSubTree().rootData());
        btree.inorder();
        System.out.println();
        btree.preorder();
        System.out.println();
        btree.postorder();
        System.out.println();
        btree.levelorder();
        System.out.println();
        current = btree.copy();
        current.inorder();
        System.out.println();
        if (btree.equals(current))
            System.out.println("They are the same trees");
        else
            System.out.println("They are different trees");
        if (btree.equals(ltree))
            System.out.println("They are the same trees");
        else
            System.out.println("They are different trees");
    }
}
```

5주차 / Queue.java

알고리즘 · 코드 모음

```java
package week_05;

public class Queue {
    private int front;
    private int back;
    private Object[] data;
    private final static int MAX = 100;

    public Queue() {
        data = new Object[MAX];
        front = back = 0;
    }

    public int size() {
        if (back >= front)
            return back - front;
        else
            return MAX + back - front;
    }

    public boolean isEmpty() {
        return front == back;
    }

    public void enqueue(Object x) {
        back = (back + 1) % MAX;
        if (back == front) {
        }
        data[back] = x;
    }

    public Object dequeue() {
        front = (front + 1) % MAX;
        return data[front];
    }

    public Object front() {
        return data[(front + 1) % MAX];
    }
}
```

5주차 / Stack.java

알고리즘 · 코드 모음

```java
package week_05;

public class Stack {
    private int top;
    private Object[] data;
    private final static int MAX = 100;

    public Stack() {
        data = new Object[MAX];
        top = -1;
    }

    public int size() {
        return top + 1;
    }

    public boolean empty() {
        return top == -1;
    }

    public void push(Object x) {
        top = top + 1;
        data[top] = x;
    }

    public Object pop() {
        top = top - 1;
        return data[top + 1];
    }

    public Object peek() {
        return data[top];
    }
}
```

7주차 / BSTTest.java

알고리즘 · 코드 모음

```java
package week_07;

public class BSTTest {
    public static void main(String[] args) {
        BinarySearchTree t = new BinarySearchTree();
        t.BSTinsert("S");
        t.BSTinsert("J");
        t.BSTinsert("B");
        t.BSTinsert("D");
        t.BSTinsert("U");
        t.BSTinsert("M");
        t.BSTinsert("R");
        t.BSTinsert("Q");
        t.BSTinsert("A");
        t.BSTinsert("G");
        t.BSTinsert("E");
        System.out.println("===== 초기 이진 탐색 트리 =====");
        t.print();
        System.out.println("===== R 검색 =====");
        TreeNode n = t.BSTsearch("R");
        if (n == null)
            System.out.println("Not Found");
        else
            System.out.println("Found");
        System.out.println("===== R 삭제 후 검색 =====");
        t.BSTdelete("R");
        n = t.BSTsearch("R");
        if (n == null)
            System.out.println("Not Found");
        else
            System.out.println("Found");
        System.out.println("R 삭제 후 트리");
        t.print();
        System.out.println("===== E 검색 =====");
        n = t.BSTsearch("E");
        if (n == null)
            System.out.println("Not Found");
        else
            System.out.println("Found");
        System.out.println("===== E 삭제 후 검색 =====");
        t.BSTdelete("E");
        n = t.BSTsearch("E");
        if (n == null)
            System.out.println("Not Found");
        else
            System.out.println("Found");
        System.out.println("E 삭제 후 트리");
```

```
48          t.print();
49          System.out.println("===== J 검색 및 삭제 =====");
50          n = t.BSTsearch("J");
51          if (n == null)
52              System.out.println("Not Found");
53          else
54              System.out.println("Found");
55          t.BSTdelete("J");
56          System.out.println("J 삭제 후 검색");
57          n = t.BSTsearch("J");
58          if (n == null)
59              System.out.println("Not Found");
60          else
61              System.out.println("Found");
62          System.out.println("J 삭제 후 트리");
63          t.print();
64          System.out.println("===== S 검색 및 삭제 =====");
65          n = t.BSTsearch("S");
66          if (n == null)
67              System.out.println("Not Found");
68          else
69              System.out.println("Found");
70          t.BSTdelete("S");
71          System.out.println("S 삭제 후 검색");
72          n = t.BSTsearch("S");
73          if (n == null)
74              System.out.println("Not Found");
75          else
76              System.out.println("Found");
77          System.out.println("S 삭제 후 트리");
78          t.print();
79          System.out.println("===== B 검색 및 삭제 =====");
80          n = t.BSTsearch("B");
81          if (n == null)
82              System.out.println("Not Found");
83          else
84              System.out.println("Found");
85          t.BSTdelete("B");
86          System.out.println("B 삭제 후 검색");
87          n = t.BSTsearch("B");
88          if (n == null)
89              System.out.println("Not Found");
90          else
91              System.out.println("Found");
92          System.out.println("B 삭제 후 트리");
93          t.print();
94          System.out.println("===== 존재하지 않는 C 검색 =====");
```

```
95          n = t.BSTsearch("C");
96          if (n == null)
97              System.out.println("Not Found");
98          else
99              System.out.println("Found");
100         System.out.println("===== 새로운 이진 탐색 트리 t1 생성 =====");
101         BinarySearchTree t1 = new BinarySearchTree();
102         t1.BSTinsert("S");
103         t1.BSTinsert("J");
104         t1.BSTinsert("B");
105         t1.BSTinsert("D");
106         t1.BSTinsert("U");
107         t1.BSTinsert("M");
108         t1.BSTinsert("R");
109         t1.BSTinsert("Q");
110         t1.BSTinsert("A");
111         t1.BSTinsert("G");
112         t1.BSTinsert("E");
113         System.out.println("t1 트리");
114         t1.print();
115         System.out.println("===== t1에서 R 검색 =====");
116         n = t1.BSTsearch("R");
117         if (n == null)
118             System.out.println("Not Found");
119         else
120             System.out.println("Found");
121         System.out.println("===== t1에서 C 검색 =====");
122         n = t1.BSTsearch("C");
123         if (n == null)
124             System.out.println("Not Found");
125         else
126             System.out.println("Found");
127         BinarySearchTree Small = new BinarySearchTree();
128         BinarySearchTree Large = new BinarySearchTree();
129         System.out.println("===== t 트리를 D를 기준으로 분할 =====");
130         t.BSTsplit("D", Small, Large);
131         System.out.println("D보다 작은 트리");
132         Small.print();
133         System.out.println("D보다 큰 트리");
134         Large.print();
135         System.out.println("===== t1 트리를 C를 기준으로 분할 =====");
136         t1.BSTsplit("C", Small, Large);
137         System.out.println("C보다 작은 트리");
138         Small.print();
139         System.out.println("C보다 큰 트리");
140         Large.print();
141         System.out.println("===== T를 루트로 하는 새로운 트리 생성 =====");
```

```
142         t1 = new BinarySearchTree();
143         t1.BSTinsert("T");
144         t1.BSTinsert("U");
145         t1.BSTinsert("S");
146         t1.BSTinsert("R");
147         t1.BSTinsert("O");
148         t1.BSTinsert("M");
149         t1.BSTinsert("W");
150         t1.BSTinsert("X");
151         t1.BSTinsert("C");
152         t1.BSTinsert("B");
153         t1.BSTinsert("A");
154         System.out.println("T 삭제 전 트리");
155         t1.print();
156         System.out.println("===== T 삭제 =====");
157         t1.BSTdelete("T");
158         System.out.println("T 삭제 후 트리");
159         t1.print();
160         System.out.println("===== X, R 삭제 테스트를 위한 새로운 트리 생성 =====");
161         t1 = new BinarySearchTree();
162         t1.BSTinsert("T");
163         t1.BSTinsert("U");
164         t1.BSTinsert("R");
165         t1.BSTinsert("S");
166         t1.BSTinsert("O");
167         t1.BSTinsert("M");
168         t1.BSTinsert("X");
169         t1.BSTinsert("W");
170         t1.BSTinsert("C");
171         t1.BSTinsert("B");
172         t1.BSTinsert("Y");
173         System.out.println("삭제 전 트리");
174         t1.print();
175         System.out.println("===== X 삭제 =====");
176         t1.BSTdelete("X");
177         System.out.println("X 삭제 후 트리");
178         t1.print();
179         System.out.println("===== R 삭제 =====");
180         t1.BSTdelete("R");
181         System.out.println("R 삭제 후 트리");
182         t1.print();
183     }
184 }
```

7주차 / BinarySearchTree.java

알고리즘 · 코드 모음

```java
package week_07;

public class BinarySearchTree {
    private TreeNode rootNode;

    public TreeNode BSTsearch(String K) {
        // 코드 작성
        TreeNode p = rootNode;

        while (p != null) {
            int result = K.compareTo(p.key);

            if (result == 0)
                return p;
            else if (result < 0)
                p = p.Lchild;
            else
                p = p.Rchild;
        }

        return null;
    }

    public void BSTinsert(String K) {
        // 코드 작성
        TreeNode newNode = new TreeNode();
        newNode.key = K;
        newNode.Lchild = null;
        newNode.Rchild = null;

        if (rootNode == null) {
            rootNode = newNode;
            return;
        }

        TreeNode p = rootNode;
        TreeNode parent = null;

        while (p != null) {
            parent = p;

            int result = K.compareTo(p.key);

            if (result == 0)
                return;
            else if (result < 0)
                p = p.Lchild;
```

```
            else
                p = p.Rchild;
        }

        if (K.compareTo(parent.key) < 0)
            parent.Lchild = newNode;
        else
            parent.Rchild = newNode;
    }

    private TreeNode maxNode(TreeNode root) {
        // 코드 작성
        if (root == null)
            return null;

        while (root.Rchild != null) {
            root = root.Rchild;
        }

        return root;
    }

    private TreeNode delete(TreeNode root, String K) {
        // 코드 작성
        if (root == null)
            return null;

        int result = K.compareTo(root.key);

        if (result < 0) {
            root.Lchild = delete(root.Lchild, K);
        } else if (result > 0) {
            root.Rchild = delete(root.Rchild, K);
        } else {
            // 자식이 없는 경우 또는 오른쪽 자식만 있는 경우
            if (root.Lchild == null)
                return root.Rchild;

            // 왼쪽 자식만 있는 경우
            if (root.Rchild == null)
                return root.Lchild;

            // 자식이 두 개 있는 경우
            TreeNode temp = maxNode(root.Lchild);
            root.key = temp.key;
            root.Lchild = delete(root.Lchild, temp.key);
        }
```

```
95
96          return root;
97      }
98
99      public void BSTdelete(String K) {
100         rootNode = delete(rootNode, K);
101     }
102
103     // Split by relinking existing nodes along one search path: O(h).
104     // SmallTree, LargeTree and this must be distinct objects.
105     // After splitting, this tree is empty; the nodes belong to the outputs.
106     public boolean BSTsplit(String x, BinarySearchTree SmallTree,
107                             BinarySearchTree LargeTree) {
108         if (x == null || SmallTree == null || LargeTree == null)
109             throw new IllegalArgumentException("Arguments must not be null");
110         if (SmallTree == this || LargeTree == this || SmallTree == LargeTree)
111             throw new IllegalArgumentException("Trees must be distinct");
112
113         TreeNode smallDummy = new TreeNode();
114         TreeNode largeDummy = new TreeNode();
115         TreeNode s = smallDummy;
116         TreeNode l = largeDummy;
117         TreeNode p = rootNode;
118         boolean found = false;
119
120         while (p != null) {
121             int result = x.compareTo(p.key);
122             if (result == 0) {
123                 s.Rchild = p.Lchild;
124                 l.Lchild = p.Rchild;
125                 p.Lchild = null;
126                 p.Rchild = null;
127                 found = true;
128                 break;
129             } else if (result < 0) {
130                 // p and its right subtree belong to LargeTree.
131                 l.Lchild = p;
132                 l = p;
133                 p = p.Lchild;
134             } else {
135                 // p and its left subtree belong to SmallTree.
136                 s.Rchild = p;
137                 s = p;
138                 p = p.Rchild;
139             }
140         }
141
```

```
142             if (!found) {
143                 // Cut stale links when x does not exist.
144                 s.Rchild = null;
145                 l.Lchild = null;
146             }
147             SmallTree.rootNode = smallDummy.Rchild;
148             LargeTree.rootNode = largeDummy.Lchild;
149             rootNode = null;
150             return found;
151         }
152 
153         private void printNode(TreeNode n) {
154             if (n != null) {
155                 System.out.print("(");
156                 printNode(n.Lchild);
157                 System.out.print(" " + n.key + " ");
158                 printNode(n.Rchild);
159                 System.out.print(")");
160             }
161         }
162 
163         public void print() {
164             printNode(rootNode);
165             System.out.println();
166         }
167 }
```

7주차 / TreeNode.java

알고리즘 · 코드 모음

```java
package week_07;

public class TreeNode {
    String key;
    TreeNode Lchild;
    TreeNode Rchild;
}
```